

DOCUMENTAÇÃO TÉCNICA

VVM Advocacia

Arquitetura, código e infraestrutura do site institucional e blog de Vanessa Vaz Marschallinger

Versão	Data	Domínio	Repositório
1.0 — MVP Completo	Junho 2026	vanessavazadv.com.br	JustinoCarneiro/vanessa_vaz_advogada

Sumário

01	Visão Geral do Projeto	3
02	Stack e Versões	4
03	Infraestrutura em Produção	5
04	Arquitetura do Sistema	6
05	Estrutura de Arquivos	7
06	Payload CMS — Coleções e Globals	9
07	Schema do Banco de Dados	10
08	Páginas e Rotas	11
09	Camada de API e Cache	12
10	Formulário de Contato	13
11	Armazenamento de Mídia	13
12	Identidade Visual	14
13	Variáveis de Ambiente	15
14	Deploy e CI/CD	16
15	Quirks Críticos	16
16	Troubleshooting	17

1 Visão Geral do Projeto

Site institucional com blog integrado para **Vanessa Vaz Marschallinger**, advogada especializada em Direito Previdenciário. Baseada em Gunskirchen, Áustria, ela atende brasileiros no Brasil e em mais de 20 países via acordos bilaterais e multilaterais com a Previdência Social.



Objetivo

Presença digital profissional com geração de leads via formulário de contato e blog de conteúdo para SEO orgânico.



Usuária do CMS

Vanessa — não-técnica. O painel deve permitir publicar posts, atualizar fotos e editar serviços sem suporte técnico.



Visitantes

Mobile-first. Brasileiros no exterior e no Brasil que buscam orientação previdenciária.



Performance

LCP < 2.5s, imagens via next/image, cache de 60s com revalidação por tag, CDN global via Vercel.

Páginas do site

Rota	Página	Renderização
/	Home	SSR (dados do CMS cacheados)
/sobre	Sobre Mim	SSR
/escritorio	Sobre o Escritório	SSR
/servicos	Serviços	SSR (cards do CMS)
/brasileiros-no-exterior	Brasileiros no Exterior	SSR
/blog	Blog — listagem	SSR com busca e filtro por categoria
/blog/[slug]	Artigo individual	SSR com revalidação em 60s
/blog/categoria/[slug]	Posts por categoria	SSR
/contato	Formulário de contato	SSR + Client (form)
/admin	Painel CMS Payload	Client-side (SPA)

2 Stack e Versões

Camada	Tecnologia	Versão	Função
Framework	Next.js	15.4.11	App Router, SSR/SSG, API routes
Runtime	React	19.1.2	Renderização de componentes
CMS	Payload CMS	3.x	Admin panel, coleções, globals, API
DB adapter	@payloadcms/db-postgres	3.x	Drizzle + PostgreSQL
Rich text	@payloadcms/richtext-lexical	3.x	Editor de artigos do blog
Mídia	@payloadcms/storage-vercel-blob	3.85.1+	Upload de imagens para CDN
Estilização	Tailwind CSS	3.4.x	Utility-first CSS
Email	Nodemailer	9.x	Gmail SMTP — emails do formulário
Imagens	sharp	0.33.x	Processamento e redimensionamento
Banco	PostgreSQL via Neon	—	Dados do CMS (serverless)
Deploy	Vercel	—	Hosting, CDN, Blob, CI/CD
Testes unit.	Jest + ts-jest	29.x	Testes unitários
Testes E2E	Playwright	1.60.x	Testes de navegação no browser
Node.js	—	20+ (prod: 24)	Runtime

3 Infraestrutura em Produção

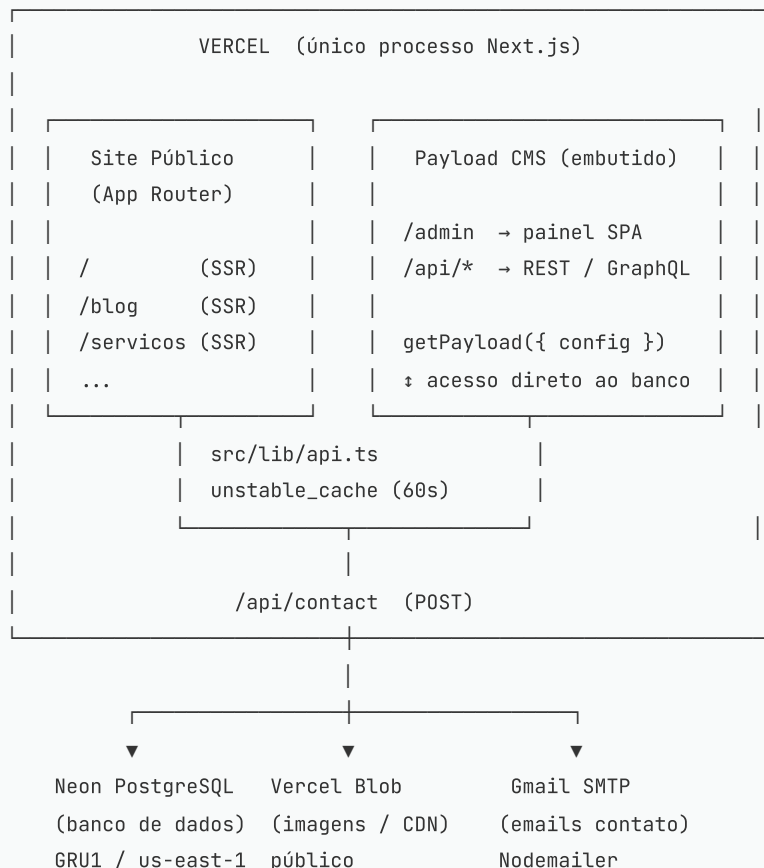
PROPRIEDADE

Toda a infraestrutura está na conta da cliente (vazvanessamarschallinger@gmail.com). O desenvolvedor não tem acesso sem as credenciais dela. Migração realizada em junho de 2026.

Serviço	Recurso / identificador	Plano
Vercel	Projeto: <code>vanessa-vaz-advogada</code>	Hobby (gratuito)
Neon (PostgreSQL)	<code>ep-broad-thunder-ais0lkb2.c-4.us-east-1.aws.neon.tech</code>	Free (0.5 GB)
Vercel Blob	Store: <code>store_TogqL7TC60PxomF4</code> (vanessa-vaz-advogada-blob)	Hobby incluso
Gmail SMTP	vazvanessamarschallinger@gmail.com — App password configurada	Gratuito
Domínio	<code>vanessavazadv.com.br</code> apontando para Vercel	—
GitHub	<code>JustinoCarneiro/vanessa_vaz_advogada</code>	—

Fluxo de deploy

```
Developer (push na branch main)
  |
  ▼
GitHub — JustinoCarneiro/vanessa_vaz_advogada
  |   webhook automático
  ▼
Vercel CI/CD
├─ npm install
├─ next build (TypeScript, Tailwind, Payload)
└─ deploy para produção
    |
    ▼
vanessavazadv.com.br (CDN Vercel, região GRU1)
├─ Site público (Next.js SSR/SSG)
├─ Painel CMS (/admin — Payload)
└─ API routes (/api/contact)
```



Payload CMS embutido no Next.js

O Payload 3.x roda dentro do mesmo processo Next.js, sem servidor separado. As rotas `/admin` e `/api` são servidas como route groups `(payload)`. Resultado: um único deploy no Vercel cobre site público e painel CMS, sem custo adicional.

Schema do banco — `push: true`

O adapter PostgreSQL usa `push: true`, que sincroniza o schema automaticamente no startup — equivalente ao `db push` do Prisma. **Não existem arquivos de migration manuais.**

ATENÇÃO

Se o banco ficar com schema inconsistente após migração manual de dados, a solução é dropar todas as tabelas e reiniciar o servidor — o Payload recria tudo corretamente via `push: true`. Ver Seção 15 (Quirks).

```
vanessa-vaz-projeto/
├─ src/
│  └─ app/
│     └─ (site)/
│        # Grupo: rotas públicas
│        └─ layout.tsx      # NavBar + Footer + NavigationProgress
│        └─ page.tsx       # Home: hero, bio, serviços, CTA, blog
│        └─ sobre/page.tsx  # Sobre Mim
│        └─ escritorio/page.tsx # Sobre o Escritório
│        └─ servicos/page.tsx # Cards de serviços do CMS
│        └─ brasileiros-no-exterior/ # Página especializada
│        └─ contato/page.tsx  # Formulário de contato
│        └─ blog/
│           └─ page.tsx      # Listagem com busca e filtro
│           └─ [slug]/page.tsx # Artigo individual (rich text)
│           └─ categoria/[slug]/ # Posts por categoria
│
│     └─ (payload)/
│        # Grupo: rotas internas do Payload
│        └─ admin/
│           └─ importMap.js  # ⚠ EDITADO MANUALMENTE – não gerar
│           └─ [...segments]/ # Catch-all do admin SPA
│        └─ api/[...slug]/route.ts # REST / GraphQL do Payload
│        └─ layout.tsx
│
│     └─ api/contact/route.ts # POST: valida, salva e envia email
│     └─ sitemap.ts          # Sitemap dinâmico
│     └─ robots.ts
│     └─ globals.css         # Reset, variáveis CSS, @font-face
│     └─ layout.tsx          # Root layout: fontes, metadata, favicon
```

```

|   |─ collections/                # Schemas das coleções Payload
|   |   |─ Users.ts                # Admin único (email + senha bcrypt)
|   |   |─ Posts.ts               # Blog: título, slug, rich text, capa
|   |   |─ Categories.ts          # Categorias com slug auto-gerado
|   |   |─ ContactMessages.ts     # Mensagens do formulário (read: bool)
|   |   |─ Media.ts               # Uploads → Vercel Blob em produção
|   |
|   |─ globals/
|   |   |─ SiteSettings.ts        # sobreFoto, escritorioFoto, servicos[]
|   |
|   |─ components/
|   |   |─ blog/
|   |   |   |─ PostCard.tsx       # Card: capa, categoria, título, excerpt
|   |   |   |─ SearchBar.tsx     # Busca client (?q= na URL)
|   |   |   |─ CategoryPills.tsx  # Filtro client (?categoria=)
|   |   |   |─ RichTextRenderer.tsx # Lexical JSON → HTML semântico
|   |   |─ contact/ContactForm.tsx # Form com validação + fetch /api/contact
|   |   |─ home/
|   |   |   |─ HeroCarousel.tsx   # 3 slides automáticos (client)
|   |   |   |─ ServiceCard.tsx    # Card simples (home – ícone+título+desc)
|   |   |   |─ CtaSection.tsx     # Chamada para agendamento
|   |   |   |─ ExteriorSection.tsx # Bloco sobre atendimento internacional
|   |   |─ servicos/
|   |   |   |─ ServicosCards.tsx  # Cards expandidos com bullets + botão
|   |   |   |─ servicosIcons.tsx  # Mapa slug → SVG + ICONES_OPTIONS
|   |   |─ layout/
|   |   |   |─ NavBar.tsx         # Navbar responsiva com menu mobile
|   |   |   |─ Footer.tsx        # Links, redes sociais, LinkedIn
|   |   |   |─ NavigationProgress.tsx # Barra de progresso entre páginas
|   |   |─ seo/JsonLd.tsx        # Schema.org JSON-LD (LegalService)
|   |   |─ ui/
|   |   |   |─ Button.tsx         # Variantes: primary / outline
|   |   |   |─ Container.tsx      # max-width + padding responsivo
|   |   |   |─ Logo.tsx          # Logo com variantes de cor
|   |   |   |─ SectionTitle.tsx   # Título de seção com overline
|   |
|   |─ lib/api.ts                 # getPosts, getPostBySlug, getSiteSettings
|   |─ __tests__/                 # Jest por módulo (m01-m07)
|
|─ scripts/                       # Utilitários Node.js (ESM)
|   |─ seed.mjs                   # Popula banco com dados de exemplo
|   |─ init-settings.mjs          # Inicializa SiteSettings
|   |─ upload-fotos.mjs           # Upload fotos para o Blob
|   |─ generate-og.mjs            # Gera imagens Open Graph estáticas
|
|─ design/                         # Identidade visual da cliente
|   |─ tokens.css                 # Variáveis de cor, tipografia, espaçamento
|   |─ DESIGN.md                  # Manual resumido da marca
|
|─ docs/                           # Documentação técnica
|─ payload.config.ts               # Configuração central do Payload
|─ next.config.ts                  # remotePatterns, headers, etc.
|─ tailwind.config.ts              # Tema com cores e fontes da marca

```



```
|— tsconfig.json / jest.config.ts / playwright.config.ts
```

```
|— .env.example
```

```
# Template de variáveis de ambiente
```

6 Payload CMS — Coleções e Globals

Configurado em `payload.config.ts`. Define coleções (tabelas com CRUD no admin) e globals (registros únicos de configuração).

Coleções

Slug	Tabela	Campos principais	Acesso
<code>users</code>	users	email, hash (bcrypt), salt, name	Admin autenticado
<code>posts</code>	posts	title, slug (auto), status (draft/published), excerpt, content (Lexical JSON), coverImage, category, publishedAt	Leitura pública; escrita admin
<code>categories</code>	categories	name, slug (auto-gerado)	Leitura pública; escrita admin
<code>contact-messages</code>	contact_messages	name, email, phone, subject, message, read (boolean), createdAt	Criação via API pública; leitura admin
<code>media</code>	media	filename, mimeType, filesize, width, height, url, alt	Upload admin; URL pública via CDN

Global — SiteSettings (slug: site-settings)

Label: **"Fotos e Serviços"**. Registro único com configurações editáveis pela Vanessa no painel.

Campo	Tipo	Uso no site
sobreFoto	upload → media	Foto da advogada (home + /sobre)
escritorioFoto	upload → media	Foto do escritório (/escritorio)
servicos[]	array (máx. 7)	Cards de serviços em /servicos e na home
servicos[].icone	select	Slug do ícone SVG (mapeado em servicosIcons.tsx)
servicos[].titulo	text	Título do card de serviço
servicos[].descricao	textarea	Descrição do serviço
servicos[].itens[]	array (máx. 5)	Bullets de destaque do serviço

Trecho de payload.config.ts

```
export default buildConfig({
  serverURL: process.env.NEXT_PUBLIC_SERVER_URL,
  collections: [Users, Posts, Categories, ContactMessages, Media],
  globals: [SiteSettings],
  db: postgresAdapter({
    pool: { connectionString: process.env.DATABASE_URI },
    push: true, // sincroniza schema no startup – sem migrations manuais
  }),
  editor: lexicalEditor(),
  plugins: [
    // Plugin Blob só ativo quando token está presente (produção)
    ...(process.env.BLOB_READ_WRITE_TOKEN ? [
      vercelBlobStorage({
        enabled: true,
        collections: { media: { disablePayloadAccessControl: true } },
        token: process.env.BLOB_READ_WRITE_TOKEN,
      })
    ] : []),
  ],
  secret: process.env.PAYLOAD_SECRET,
})
```

7 Schema do Banco de Dados

Schema gerado e mantido automaticamente pelo Payload. Banco hospedado no Neon (PostgreSQL serverless), região GRU1.

Tabela	Gerada por	Descrição
<code>users</code>	Coleção Users	Admin único. Senha em bcrypt (hash + salt separados).
<code>users_sessions</code>	Payload interno	Sessões JWT ativas do admin.
<code>posts</code>	Coleção Posts	Artigos do blog com status, slug e Lexical JSON.
<code>categories</code>	Coleção Categories	Categorias com slug único.
<code>media</code>	Coleção Media	Metadados de uploads. URL aponta para Vercel Blob em prod.
<code>contact_messages</code>	Coleção ContactMessages	Mensagens do formulário. Campo <code>read</code> (boolean) controla lidos no admin.
<code>site_settings</code>	Global SiteSettings	Registro único com as fotos e array de serviços.
<code>site_settings_servicos</code>	Array field	Cada serviço do global. <code>_order</code> define a ordem de exibição.
<code>site_settings_servicos_itens</code>	Array aninhado	Bullets de cada serviço. <code>_order</code> define a ordem.
<code>payload_preferences</code>	Payload interno	Preferências do admin (tema, colunas, etc.).
<code>payload_preferences_rels</code>	Payload interno	Relações das preferências.
<code>payload_locked_documents</code>	Payload interno	Controle de edição concorrente.
<code>payload_locked_documents_rels</code>	Payload interno	Relações dos locks.
<code>payload_migrations</code>	Payload interno	Histórico de migrations executadas.
<code>payload_kv</code>	Payload interno	Key-value store interno.

Enums PostgreSQL

Enum	Valores
enum_posts_status	draft, published
enum_site_settings_servicos_icone	relogio, escudo, cadeado, pessoa, pessoas, seta-circular, globo, pasta, balanca

Home — `src/app/(site)/page.tsx`

Renderização SSR. Busca em paralelo: posts recentes (3) + SiteSettings (foto + serviços). Seções:

- **HeroCarousel** — 3 slides com transição automática. Texto e foto fixos no código (client component).
- **Quem é Vanessa** — foto vinda do CMS (`sobreFoto`) + bio estática.
- **Áreas de Atuação** — grid de `ServiceCard` com dados do CMS (`servicos`).
- **ExteriorSection** — bloco fixo sobre brasileiros no exterior.
- **CtaSection** — chamada para agendamento de consulta.
- **Blog** — últimos 3 posts publicados com fallback de "em breve".

Blog — `src/app/(site)/blog/`

Listagem com busca por texto (`?q=`) e filtro por categoria (`?categoria=`) via query params na URL, lidos no Server Component. Artigo individual com `generateMetadata` dinâmico para SEO e `RichTextRenderer` para o conteúdo Lexical JSON.

Serviços — `src/app/(site)/servicos/page.tsx`

Busca o global `SiteSettings` e renderiza `ServicosCards` — versão expandida com bullets e botão "Fale sobre este caso" que leva ao formulário com assunto pré-preenchido via query param (`?assunto=`).

API de contato — `src/app/api/contact/route.ts`

Método POST. Fluxo completo:

1. Parse e validação dos campos (nome, email, telefone opcional, assunto, mensagem mín. 20 chars)
2. Salva em `contact-messages` via Payload (independente do email)
3. Envia email via Gmail SMTP — falha silenciosamente se variáveis ausentes
4. Retorna `{ ok: true }` em sucesso ou `{ errors }` em validação

Toda comunicação com o Payload nas páginas SSR passa por `src/lib/api.ts`. As funções usam `unstable_cache` do Next.js.

Função	Cache key	Revalidação	Tags
<code>getPosts(options)</code>	posts, page-N, limit-N	60s	posts
<code>getPostBySlug(slug)</code>	post, {slug}	60s	posts, post-{slug}
<code>getCategories()</code>	categories	300s	categories
<code>getSiteSettings()</code>	site-settings	60s	site-settings

BUSCAS FILTRADAS NÃO SÃO CACHEADAS

Quando `getPosts` recebe `query` ou `category`, bypassa o cache para evitar explosão de cache keys (uma por combinação de parâmetros). A busca vai diretamente ao Payload/banco.

Tipos auxiliares em api.ts

Para evitar depender do `payload generate:types`, os tipos mínimos são definidos manualmente:

```
export type SiteSettingsServico = {
  icone?: string | null
  titulo: string
  descricao: string
  itens?: Array<{ item: string; id?: string }>
  id?: string
}

// Resolve o campo de upload que pode ser objeto ou ID numérico
export function extractMediaUrl(
  field: SiteSettingsMedia | number | null | undefined
): string | null {
  if (!field || typeof field !== 'number') return null
  return field.url ?? null
}
```

10 Formulário de Contato

O `ContactForm.tsx` é um client component com validação local sem biblioteca externa. Ao submeter, faz `fetch POST` para `/api/contact` e exibe feedback de sucesso ou erros por campo.

Campo	Obrigatório	Validação
Nome	Sim	Não vazio
Email	Sim	Regex básico de email
Telefone	Não	Se preenchido: DDD + número (10 ou 11 dígitos)
Assunto	Sim	Select com opções pré-definidas
Mensagem	Sim	Mínimo 20 caracteres

O email usa o Gmail da cliente como remetente (`EMAIL_USER`), com `Reply-To` apontando para o email do visitante — assim Vanessa responde diretamente ao cliente ao clicar em "Responder" no Gmail.

Mesmo que o envio de email falhe (ex: variáveis não configuradas), a mensagem é salva na coleção `contact-messages` e fica visível no painel `/admin`.

11 Armazenamento de Mídia

Ambiente	Comportamento
Desenvolvimento (sem token)	Uploads em memória/disco local. Sem persistência entre reinicializações.
Produção (com token)	Plugin <code>@payloadcms/storage-vercel-blob</code> intercepta uploads e envia para o Blob CDN. URLs apontam diretamente para <code>*.public.blob.vercel-storage.com</code> sem proxy.

O `next.config.ts` inclui `remotePatterns` para o domínio do Blob, permitindo que `next/image` otimize essas imagens. O `sharp` gera versões redimensionadas automaticamente no upload.

STORE EM PRODUÇÃO

ID: `store_TogqL7TC60PxomF4` · Nome: vanessa-vaz-advogada-blob · Região: GRU1 · Acesso: público

A cliente possui identidade visual fechada. Assets em `design/tokens.css` e `design/DESIGN.md`. A implementação segue o manual da marca sem criação de identidade do zero.

Paleta de cores

Token	Hex	Uso
Teal escuro (primária)	#3D5C5F	Botões, links, bordas, headings, fundo hero
Teal médio	#89B0AF	Elementos secundários, ícones, dots do carrossel
Menta	#BDE4DA	Fundos de seção, fundo de ícones, detalhe decorativo
Preto texto	#111111	Títulos principais (h1, h2, h3)
Cinza corpo	#333333	Parágrafos, texto de cards
Cinza secundário	#666666	Labels, metadados, datas
Linha divisória	#E4EAE9	Bordas de cards, separadores

Tipografia

Fonte	Pesos	Uso
Cormorant Garamond	400, 500, 600, 700	Display: títulos de seção, hero H1, H2 de destaque
Montserrat	300, 400, 500, 600, 700	Corpo: parágrafos, labels, botões, NavBar

Carregadas via Google Fonts no `src/app/layout.tsx`. A fonte display é referenciada como `var(--font-display)` via CSS variable configurada pelo Next.js.

Logo e assets

Símbolo VM circular + logotipo horizontal. Variantes: dourado, teal, preto, branco. Assets em `src/assets/`. O componente `Logo.tsx` seleciona a variante por prop. Marca d'água disponível em `src/assets/marcadagua-branco.png` e `marcadagua-preto.png`.

SEGURANÇA

O arquivo `.env.local` nunca deve ser commitado (está no `.gitignore`). Em produção, as variáveis são gerenciadas pelo Vercel em Settings → Environment Variables.

Variável	Obrig.	Descrição
<code>DATABASE_URI</code>	Sim	Connection string Neon PostgreSQL. Sempre incluir <code>?uselibpqcompat=true&sslmode=require</code> no final.
<code>PAYLOAD_SECRET</code>	Sim	Chave JWT do Payload (mín. 32 chars). Gerar: <code>openssl rand -base64 32</code> . Não trocar em produção — invalida todas as sessões.
<code>NEXT_PUBLIC_SERVER_URL</code>	Sim	URL pública sem barra final. Dev: <code>http://localhost:3000</code> . Prod: <code>https://vanessavazadv.com.br</code> .
<code>BLOB_READ_WRITE_TOKEN</code>	Prod	Token Vercel Blob. Gerado automaticamente ao criar o Blob Store. Sem ele, o plugin Blob não é ativado.
<code>EMAIL_USER</code>	Prod	Email Gmail remetente dos emails de contato da cliente.
<code>EMAIL_APP_PASSWORD</code>	Prod	Senha de app Gmail (16 chars). Google Conta → Segurança → Senhas de app.
<code>CONTACT_EMAIL_TO</code>	Não	Destinatário dos emails. Se vazio, usa o valor de <code>EMAIL_USER</code> .
<code>PLAYWRIGHT_ADMIN_EMAIL</code>	Testes	Email do admin para testes E2E Playwright.
<code>PLAYWRIGHT_ADMIN_PASSWORD</code>	Testes	Senha do admin para testes E2E Playwright.

DESENVOLVIMENTO APONTA PARA PRODUÇÃO

O `.env.local` atual usa o banco e Blob da Vanessa (produção). Alterações pelo painel local (`localhost:3000/admin`) afetam os dados reais. Cuidado com operações de delete ou seed.

Push na branch `main` dispara build e deploy automático no Vercel da Vanessa. Não há CI adicional — o Vercel é o pipeline único.

Evento	Resultado
Push em <code>main</code>	Deploy em produção (<code>vanessavazadv.com.br</code>)
Push em outra branch / PR	Deploy de preview com URL temporária (<code>*.vercel.app</code>)

Scripts principais

```
npm run dev      # next dev – desenvolvimento com hot reload
npm run build    # next build – compila TS, Tailwind, Payload
npm start       # next start – serve o build de produção localmente
npm test        # Jest – testes unitários
npm run test:e2e # Playwright – testes E2E
npm run seed     # Popula banco local com dados de exemplo
npm run init-settings # Inicializa SiteSettings (fotos + serviços)
```

NUNCA EXECUTAR EM PRODUÇÃO

`npm run payload generate:importmap` — gera um importMap incorreto no Vercel e resulta em tela branca no painel /admin. O arquivo deve ser mantido manualmente. Ver Seção 15.

QUIRK #1 — IMPORTMAP.JS NUNCA DEVE SER GERADO AUTOMATICAMENTE

O arquivo `src/app/(payload)/admin/importMap.js` é mantido manualmente. O comando `payload generate:importmap` falha silenciosamente no Vercel e resulta em tela branca no painel `/admin`. Ao adicionar novos plugins ou componentes customizados ao Payload, edite o arquivo diretamente seguindo o padrão dos imports existentes (formato `import { X } from 'pacote'` com hash no alias).

QUIRK #2 — ERRO "SET DATA TYPE SERIAL" AO INICIAR

Se o banco ficou com schema inconsistente (ex: migração manual sem schema correto), o Payload pode gerar `ALTER COLUMN "id" SET DATA TYPE serial`, que é SQL inválido no PostgreSQL. Solução definitiva:

1. Dropar todas as tabelas do banco (via console do Neon ou script)
2. Rodar `npm run dev` — Payload recria o schema corretamente via `push: true`
3. Re-importar os dados via script de INSERT direto (sem DDL)

QUIRK #3 — USELIBPQCOMPAT=TRUE OBRIGATÓRIO NA CONNECTION STRING

O driver `pg` (não o serverless driver) com Neon requer este parâmetro. Sem ele, o modo SSL pode ser incompatível. Sempre usar o formato completo:

```
postgresql://user:pass@host/db?uselibpqcompat=true&sslmode=require
```

QUIRK #4 — BANCO DE DESENVOLVIMENTO APONTA PARA PRODUÇÃO

O `.env.local` atual usa o banco e Blob da Vanessa para que o desenvolvimento espelhe o ambiente real. Operações destrutivas (delete, truncate) no painel local afetam dados de produção.

Problema	Causa provável	Solução
Painel <code>/admin</code> com tela branca	<code>importMap.js</code> desatualizado ou gerado automaticamente	Restaurar manualmente a partir do último commit funcional
Posts ou serviços não aparecem	<code>DATABASE_URI</code> incorreta ou banco vazio	Verificar env vars no Vercel → Settings → Environment Variables
Email de contato não chega	<code>EMAIL_APP_PASSWORD</code> expirada ou variável ausente	Regenerar senha de app no Google. Verificar variáveis. Checar pasta de spam.
Upload de foto falha em produção	<code>BLOB_READ_WRITE_TOKEN</code> inválido ou ausente	Verificar token no Vercel → Storage → vanessa-vaz-advogada-blob → Settings
Build falha no Vercel	Erro TypeScript ou variável env ausente no build	Ver Build Logs: Vercel → Deployments → último deploy → Build Logs
Erro "serial" ao iniciar servidor	Schema inconsistente no banco	Dropar tabelas + reiniciar (ver Quirk #2 na seção anterior)
Imagens não aparecem no site	URL do Blob incorreta ou <code>remotePatterns</code> ausente no Next.js	Verificar <code>next.config.ts</code> e campo <code>url</code> na tabela <code>media</code>
Domínio não resolve	DNS não configurado ou propagação pendente	Verificar no registrador do domínio. Usar <code>dig vanessavazadv.com.br</code> . Aguardar até 48h.

Onde encontrar logs e dados

O quê	Onde
Build logs	Vercel → projeto → Deployments → clique no deploy → Build Logs
Runtime logs	Vercel → projeto → Logs (aba superior)
Banco de dados	console.neon.tech → conta da Vanessa → SQL Editor
Blob Store	vercel.com (conta Vanessa) → Storage → vanessa-vaz-advogada-blob
Emails enviados	Gmail da Vanessa (vazvanessamarschallinger@gmail.com) → Enviados
Mensagens de contato	vanessavazadv.com.br/admin → Contact Messages

